Digital Image Processing Lab (NCSH 302) **Session: Monsoon 2026-27**
B. Tech. 5th Sem (CSE-Hons) **Date: 13.10.2026**

Assignment-9 (Image Segmentation-II)

1. Consider *simple_image.png* as the $8-bit$ grayscale input image $f(x,y)$ of size 192×192 with top-left pixel $(0,0)$. In all parts use the homogeniety property $P(R) = TRUE$ if $max\{f(x,y)\} - min\{f(x,y)\} \leq T$ over all pixels of region R, else $FALSE$, with $T = 40$.

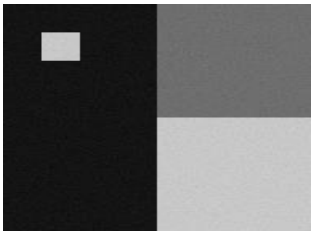
PART A – Region Splitting. Start with the whole image as one region. If P is TRUE, keep the region as final; if P is FALSE, split it into four equal quadrants by halving its height and width (quadtree). Repeat on every new region until P is TRUE for all regions. Do not merge any regions. Display the output image with labels (each region with different intensity level) and print the number of final regions.

[**5 Marks**]

PART B – Region Adjacency Graph. Build the RAG of the final regions of Part B: one node per region, and an edge between two regions R_i and R_j that share a common border, i.e., a pixel of one is a 4-neighbour of a pixel of the other (touching only at a corner is not adjacent). For every edge, test $P(R_i \cup R_j)$ over all pixels of the two original Part B regions and remove the edge if it is FALSE. Each remaining disconnected sub-graph (including a node with no edge) is one final region. Display the final label image and print the number of nodes, edges, edges removed and final regions.

[**10 Marks**]

Sample Input:



Sample Output:



Input image **PART-A** Region splitting **PART-B** After RAG

PART A: Number of regions after splitting = 10
PART B: RAG nodes = 10, edges = 16, edges removed = 8
PART B: Number of final regions after RAG = 4

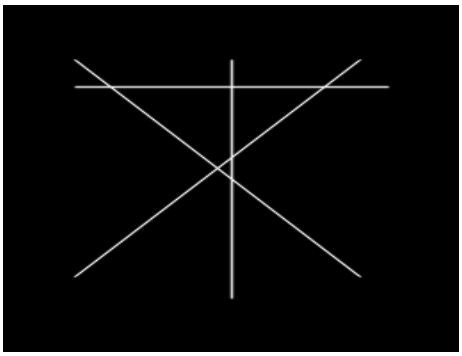
2. **PART A -Single pixel width line detection:** Read **lines_w1.png** as the input grayscale image f. Use the width-1 line detection kernels **discussed in class** for detecting lines of width 1. These are 3×3 masks with a central band of positive coefficients corresponding to the 1-pixel-wide line and negative coefficients on either side to suppress uniform regions. The sum of coefficients in each kernel is zero so that constant regions give zero response. Refer to the class notes for the exact coefficients for the four orientations: Horizontal, +45°, Vertical, and −45°. Convolve f padded with 0 with each kernel to obtain the raw signed responses. Apply thresholding to each raw response to obtain binary images. Choose an appropriate threshold to isolate the line pixels. Do not use library functions for thresholding.

[5 Marks]

PART B -: 3 pixel width line detection: Read **peacock.png** as a grayscale image. First, smooth the image using a 3x3 averaging kernel with replicate padding. Next, extend the concept of single-pixel-width line detection to a width of 3 pixels. Design and implement suitable kernels for each of the four orientations to detect 3-pixel-wide lines in the image. Perform manual convolution using replicate padding. For each kernel, compute the convolution response on the smoothed image and take its absolute value. Finally, compute the maximum response across all four orientations and scale this maximum response to the 8-bit range of 0–255 for display purposes only. **[10 Marks]**

Sample Input:

PART A: lines_w1.png

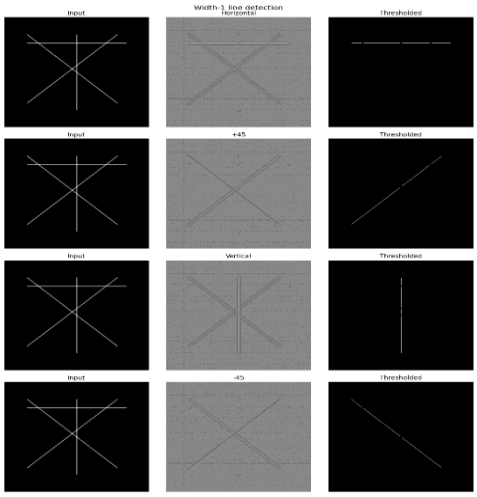


PART B: peacock.png

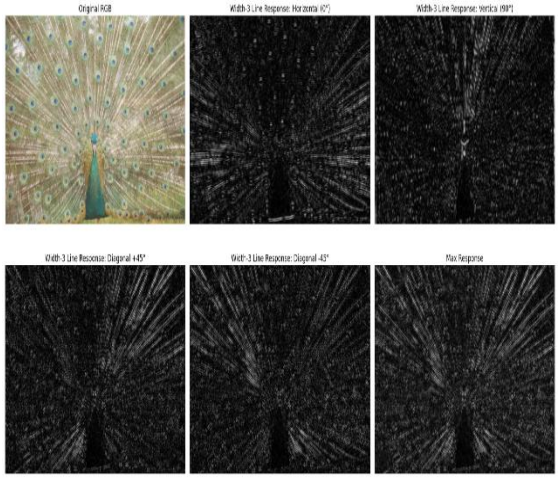


Sample Output:

PART A:



PART B



3. A satellite-based Earth observation system captures a grayscale image in which boundaries between land, ocean, and the Earth–background region appears as significant intensity transitions. The input image is provided as "earth.png". Read the image, resize the image to 512 × 512 pixels, convert it to grayscale and perform the following tasks.Apply Gaussian smoothing using the following **5 × 5** kernel. Implement the convolution manually using zero-padding.

$$G = \frac{1}{159}\begin{bmatrix} 2 & 4 & 5 & 4 & 2 \\ 4 & 9 & 12 & 9 & 4 \\ 5 & 12 & 15 & 12 & 5 \\ 4 & 9 & 12 & 9 & 4 \\ 2 & 4 & 5 & 4 & 2 \end{bmatrix}$$

Perform the convolution using zero-padding and display the Gaussian-smoothed image. After that Using the smoothed image, calculate the horizontal and vertical first-order gradient. Display the horizontal gradient $G_x = S_x * I_{smooth}$, vertical gradient $G_y = S_y * I_{smooth}$ and gradient magnitude $M(x,y) = \sqrt{G_x^2 + G_y^2}$, with proper label.
In the end Normalize the gradient magnitude to the range [0,255] using:

$$M_{(norm)(x,y)} = \frac{255(M(x,y) - M_{min})}{M_{max} - M_{min}}$$

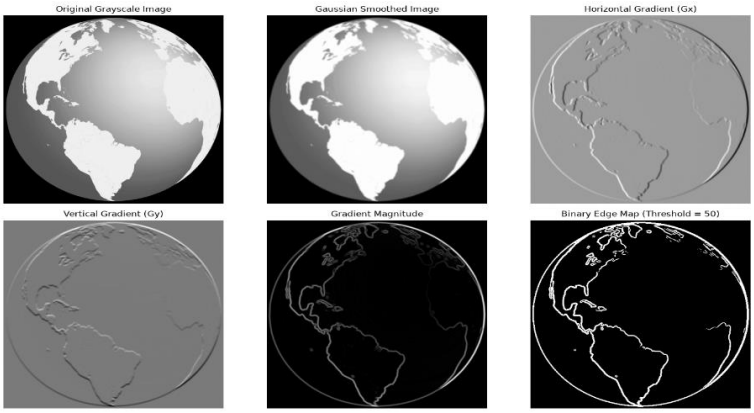
where M_{min} and M_{max} are the minimum and maximum values of the gradient magnitude image. Then generate a binary edge map using a fixed threshold T=50, Display the normalized gradient-magnitude image and the final binary edge map.

[**10 Marks**]

Sample Input:



Sample Output:



Gradient magnitude max: 425.642
Gradient magnitude 90th percentile: 36.162